Digital Image Processing

Prof. Z. Azimifar

Homework2

Topics: Image enhancement, Edge detection, and Frequency Domain Analysis

Submission Format: PDF Report + Source Code (.zip)

April 2026

Introduction and Objectives

In real-world Artificial Intelligence and Computer Vision applications, images are rarely perfect. Classical Digital Image Processing (DIP) techniques remain crucial for data preprocessing, feature extraction, and data augmentation before images are fed into Deep Learning models.

This assignment bridges classical image processing and modern algorithmic evaluation. You will implement fundamental algorithms from scratch, evaluate their performance quantitatively using ground truth data, and explore how image semantics are stored in spatial and frequency domains.

Task 1: Edge Detection & Multi-Scale Analysis

In digital images, structural information is located at regions with sharp intensity changes. In this task, you will implement classical edge detectors and evaluate them using Machine Learning metrics.

1.1 Classical First and Second-Order Derivatives

1. **Sobel Operator:** Implement discrete convolution to estimate gradients in the horizontal (G_X) and vertical (G_Y) directions. Compute the gradient magnitude:

$$G = \sqrt{G_X^2 + G_Y^2}$$

2. **Prewitt Operator:** Implement the Prewitt operator for calculating image gradients. Compare its computational structure and output to Sobel.
3. **Roberts Cross:** Implement the Roberts Cross operator using 2×2 kernels for diagonal gradient calculation.
4. **Laplacian Operator:** Implement the Laplacian operator based on the second-order derivative ($\nabla^2 f(x,y)$). You will use this for two purposes:
    - **Edge Detection:** Compute the absolute value of the Laplacian (or zero-crossings) to generate an edge map:

$$G_{edge}(x,y) = |\nabla^2 f(x,y)|.$$

    - **Image Sharpening:** Use the Laplacian to highlight regions of rapid intensity change and enhance the original image: $G_{sharp}(x,y) = f(x,y) - c \cdot \nabla^2 f(x,y)$ (where c is a constant depending on your chosen kernel's center weight).

Note: Evaluate the G_{edge} result against the Ground Truth in Task 1.3, but include the G_{sharp} image in your visual results.

1.2 The Canny Edge Detection Pipeline

Implement the complete Canny Edge Detection pipeline:

1. **Noise Reduction:** Apply a Gaussian filter.
2. **Gradient Calculation:** Use Sobel to find magnitude and direction.
3. **Non-Maximum Suppression (NMS):** Thin edges by suppressing non-peak pixels along the gradient direction.
4. **Hysteresis Thresholding:** Use adjustable high and low thresholds to trace and connect strong/weak edges.

Experiment (Multi-Scale): Run your Canny pipeline using three different sizes/variances (σ) for the initial Gaussian blur. Discuss how the scale affects the level of detail captured.

1.3 Quantitative Evaluation & Challenges

Use the images provided in the Task_1_Edge_Detection folder (e.g., 100007.jpg). Convert the images to grayscale, run your Sobel and Canny outputs, and compare them against the provided Ground Truth consensus edge map. Calculate and analyze:

- **Precision:** Ratio of correctly detected edge pixels to the total number of detected edge pixels.
- **Recall:** Ratio of correctly detected edge pixels to the total number of true edge pixels.
- **Localization Error:** Compute the average Euclidean distance between your detected edge points and the ground truth edge locations.
- **Robustness to Noise:** Evaluate how your accuracy (Precision/Recall) drops when you introduce Gaussian noise to the original image.

1.4 AI Discussion Question

How do the fixed, mathematically derived kernels (like Sobel) compare to the learnable weights in the first convolutional layer of a Convolutional Neural Network (CNN)?

Task 2: Noise Modeling & Image Restoration

Adding controlled noise to images is a standard data augmentation technique in Deep Learning. Here, you will simulate noise and attempt to restore the images using spatial filters.

2.1 Noise Generation

Take a clean grayscale image and generate degraded versions:

- Add **Gaussian noise** ($\eta \sim \mathcal{N}(0, \sigma^2)$) at three different variance levels.
- Add **Salt & Pepper noise** at three different density levels.

2.2 Image Restoration

Implement the following filters from scratch and apply them independently to all noisy versions:

- **Mean Filter**
- **Median Filter**
- **Gaussian Filter**

2.3 Quantitative Evaluation

Evaluate the performance of your restoration methods against the original clean image using two different metrics. You must implement both metrics entirely from scratch.

Part A: Mean Squared Error (MSE)

Implement a function to calculate the Mean Squared Error between the restored image and the original image.

$$MSE = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} [I_{original}(i,j) - I_{restored}(i,j)]^2$$

Part B: Structural Similarity Index Measure (SSIM)

Implement the SSIM metric. To compute local SSIM, use your custom convolution function with a uniform 11×11sliding window (or a Gaussian window) to compute local statistics. The SSIM between two image windows x and y is defined as:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

Where:

- $\mu_x, \mu_y$are the local sample means.
- $\sigma_x^2, \sigma_y^2$are the local sample variances.
- $\sigma_{xy}$is the local sample covariance.
- $C_1 = (k_1 L)^2$and $C_2 = (k_2 L)^2$, where $L = 255$(dynamic range), $k_1 = 0.01$, and $k_2 = 0.03$.

Generate a table comparing the MSE and SSIM scores across your different filters and noise types. Based on your visual results and the final scores, why is SSIM generally considered a "better" metric for image quality assessment than MSE? How do the mathematical formulas of the two metrics account for this difference in relation to human visual perception?

Task 3: Image Enhancement Using Spatial and Frequency Domain Techniques

Objective: Image enhancement refers to improving the perceptual quality of an image by highlighting structural details (edges/textures) while suppressing blur or low-frequency variations. In this task, you will analyze how sharpening operations behave in the spatial domain versus how frequency-based filtering provides an alternative interpretation of the same problem.

3.1 Spatial Domain Sharpening

The goal is to enhance fine details by manipulating pixel neighborhoods. One common approach isolates high-frequency components by subtracting a smoothed version of the image ($\bar{f}$) from the original (f).

$$g(x,y) = f(x,y) - \bar{f}(x,y)$$

where $\bar{f}(x,y)$ is the local average or a Gaussian-smoothed version of the image (you can reuse your Gaussian filter from Task 2).

A more general formulation introduces a scaling factor (k) that allows for adaptive sharpening intensity (Unsharp Masking):

$$g(x,y) = f(x,y) + k \cdot (f(x,y) - \bar{f}(x,y))$$

- **Action:** Implement this spatial sharpening. You are also expected to design and apply a simple high-pass filter using convolution kernels to directly emphasize edge structures.

3.2 Frequency Domain Representation & Algorithmic Complexity

The frequency domain decomposes an image into sinusoidal components. The 2D Discrete Fourier Transform (DFT) is defined as:

$$F(u,v) = \sum_{x} \sum_{y} f(x,y)\, e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

- **Naive DFT vs. FFT:** Briefly implement the 2D DFT using the standard mathematical formula with nested loops. Because this is $O(N^4)$, you must also implement a 1D recursive/iterative **Fast Fourier Transform (FFT)** ($O(N\log N)$), and extend it to a 2D FFT.
- **Benchmarking:** Because the Naive 2D DFT has a time complexity of O(N4), running it on a full high-resolution image will take an impractical amount of time. First, crop or resize a test image to square sizes of N = 32, 64 and 128. Measure the execution time (in seconds) of your Naive DFT vs. your 2D FFT for these specific sizes. Plot the execution times on a graph.
- **Visualization:** Compute the frequency representation using your FFT. Extract both magnitude and phase. Visualize the magnitude spectrum using a logarithmic transformation. (*Remember to center the zero-frequency (DC) component in the middle of your spectrum. You can do this by multiplying the spatial image by* $(-1)^{x+y}$ *before the transform, or by rearranging the quadrants post-transform.*)

Hint: Instead of a complex 2D recursion, use the separability property discussed in class. Apply your 1D FFT to all rows, and then apply it again to the columns of the intermediate result.

3.3 Frequency Domain Filtering & Reconstruction (IDFT)

Apply filtering operations directly on the frequency spectrum to demonstrate how different frequency components contribute to the image structure:

- **Low-pass filtering:** Implement both an **Ideal Low-pass filter** and a **Gaussian Low-pass filter**. Preserve smooth variations and remove fine details.
- **High-pass filtering:** Implement a high-pass filter to preserve edges and suppress low-frequency components.

After filtering, reconstruct the images back into the spatial domain using your **Inverse Fast Fourier Transform (IFFT)**.

3.4 Visualizing the Role of Spectrum vs. Phase

To truly understand frequency components:

1. Reconstruct an image using **only the magnitude spectrum**, setting all phase values to zero.
2. Reconstruct the image using **only the phase angle**, setting all magnitude values to one.

3.5 Required Outputs & Evaluation Metrics

For your test images, provide the following visualized outputs:

1. Original grayscale image
2. Spatially sharpened image
3. High-pass filtered image in the spatial domain
4. Frequency magnitude spectrum (log-scaled visualization)
5. Low-pass filtered reconstruction
6. High-pass filtered reconstruction

Evaluate your frequency results using:

1. **High-Frequency Energy:** A higher value indicates stronger edge content.

$$E_H = \sum_{(u,v)\in high} | F(u,v) |^2$$

2. **Spectral Energy Ratio:** The proportion of high-frequency energy to total energy.

$$R = \frac{E_{high}}{E_{total}}$$

Provide a short analysis discussing:

- Differences between spatial and frequency-domain enhancement methods.
- Compare the reconstructed outputs of the Ideal Low-pass filter vs. the Gaussian Low-pass filter. Explicitly point out and explain the "ringing" artifacts (Gibbs phenomenon).
- The effect of sharpening on edge preservation and noise amplification.
- How frequency filtering separates image information into meaningful components.
- The relationship between visual sharpness and frequency energy distribution.
- Differences between perceptual quality and numerical evaluation results.

Instructions & Deliverables

Deadline: 18 May 2026

Implementation Constraints:

- All algorithms (convolutions, edge detection pipelines, Fourier transforms) **must be implemented manually** from scratch using core mathematical operations.
- You may use numpy for basic array manipulations (e.g., slicing, matrix multiplication).
- You are **strictly prohibited** from using built-in functions like cv2.Sobel, cv2.Canny, cv2.filter2D, scipy.signal.convolve, or np.fft.*.
- Use grayscale images for all tasks unless otherwise specified.
- **Boundary Handling:** Because you are implementing convolutions manually, you must properly handle image borders to prevent dimension shrinking. You are required to implement both **Zero Padding** (padding the image borders with zeros) and **Mirror Padding** (edge replication, where the outermost pixels are duplicated). Specify in your report which padding method you used for each task and why.

Report Structure:

1. **Problem Definition:** Brief overview of objectives.
2. **Methodology & Implementation:** Explain your algorithmic approach (especially for Canny NMS and 2D FFT).
3. **Results & Visualizations:** Include original, intermediate, and final output images. Ensure frequency domain plots are log-scaled.
4. **Discussion & Quantitative Analysis:** Provide the Precision/Recall/Localization error scores, MSE tables, Execution time plots, and answer the specific theoretical discussion questions from Tasks 1.4 and 3.5.

Submission:
Submit a single .zip file containing your well-commented Python scripts (or Jupyter Notebooks) and the **PDF technical report containing all generated output images, plots, and visualizations**.

(Naming format: Student1_Student2_Student3.zip)